arXiv:2403.19851v1 [cs.CL] 28 Mar 2024

Localizing Paragraph Memorization in Language Models

Niklas Stoehr[E]* **Mitchell Gordon**[G] **Chiyuan Zhang**[G] **Owen Lewis**[G]

[E]ETH Zürich [G]Google

niklas.stoehr@inf.ethz.ch {mitchellgordon, chiyuan, lewiso}@google.com

Abstract

Can we localize the weights and mechanisms used by a language model to memorize and recite entire paragraphs of its training data? In this paper, we show that while memorization is spread across multiple layers and model components, gradients of memorized paragraphs have a distinguishable spatial pattern, being larger in lower model layers than gradients of non-memorized examples. Moreover, the memorized examples can be unlearned by fine-tuning only the high-gradient weights. We localize a low-layer attention head that appears to be especially involved in paragraph memorization. This head is predominantly focusing its attention on distinctive, rare tokens that are least frequent in a corpus-level unigram distribution. Next, we study how localized memorization is across the tokens in the prefix by perturbing tokens and measuring the caused change in the decoding. A few distinctive tokens early in a prefix can often corrupt the entire continuation. Overall, memorized continuations are not only harder to unlearn, but also to corrupt than non-memorized ones.

1 Introduction

Some language models are able to emit gigabytes of full-length paragraphs from their training data (Carlini et al., 2020, 2022; McCoy et al., 2023; Haviv et al., 2023; Nasr et al., 2023; New York Times, 2023). These memorized paragraphs must thus be represented somewhere in the model weights (Nasr et al., 2023). We take steps towards localizing these weights and internal mechanisms that are involved in the memorization of paragraphs. Specifically, we study in detail the open-weight model GPT-NEO 125M (Gao et al., 2021) which has been trained on the publicly available dataset the PILE.

*Work done while at Google

Code and data: github.com/googleinterns/localizing-paragraph-memorization

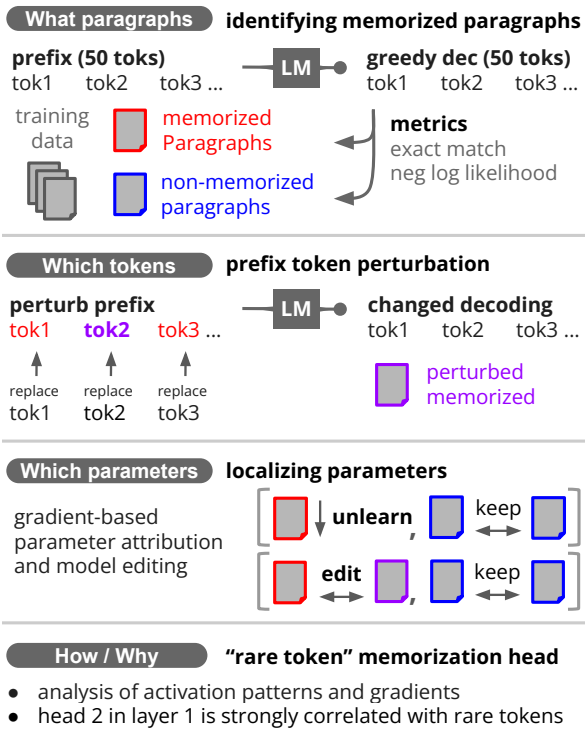

Figure 1: We interpret language models with respect to their capability to memorize 100-token paragraphs from the training data. Using sets of memorized, non-memorized as well as perturbed memorized paragraphs, we study parameter and activation gradients, activation patterns as well as unlearning and editing objectives to identify an influential "memorization head".

As a first step, we identify paragraphs that are memorized by a language model. We use the term "paragraph" for any sequence of 100 tokens. A paragraph is regarded as memorized if, given a prefix of 50 tokens, the model's greedy decoding of the next 50 tokens exactly matches the true paragraph continuation. We publish the memorized paragraphs alongside our code.

We use our dataset of memorized and non-memorized paragraphs to identify differences in how they are processed by the model. To this end, we measure the effect that perturbing individual

tokens in a paragraph's prefix has on the model's memorization. We find that "memorization triggers" can sometimes be localized to few, distinctive tokens very early in the prefix. Moreover, corrupting memorized paragraphs is, on average, more difficult than non-memorized paragraphs. The perturbed prefix continuations of previously memorized paragraphs are mostly still semantically and syntactically valid and can be regarded as alternative paraphrases.

These experiments localize "when" memorized information is accessed throughout the paragraph. To understand "where" this information may be stored, we turn to the model's parameters which are shared across all token positions. We find that parameter gradients flow indeed differently for memorized and non-memorized paragraphs. To better isolate these gradient differences, we adapt a contrastive objective from prior work (Maini et al., 2023) that seeks to reduce the likelihood of memorized paragraphs while leaving non-memorized paragraphs unchanged. This objective has the additional advantage that it can be used to (sparsely) fine-tune the model: we upgrade only those parameters that we have previously localized and validate that our localization does in fact inform editing (Hase et al., 2023). In particular, we experiment with two fine-tuning objectives, one that "unlearns" and one that "edits" memorized paragraphs into their perturbed alternatives. We find that unlearning is easier than editing, and it is often difficult to leave non-memorized paragraphs unchanged.

While memorization is spread across multiple layers and components of the model, there is one model component that is standing out: attention head 2 in layer 1. Analyzing activation gradients and attention patterns, we qualitatively and quantitatively show that this head attends predominantly to distinctive, or rare tokens in the long tail of the unigram token distribution. We include additional experiments with activation patching and activation gradients in the appendix.

2 Related Work

This paper connects three lines of work on language models: memorization, interpretability and editing.

Memorization in Language Models. Our work builds upon Carlini et al. (2022), who quantify which and how many paragraphs from the training data are memorized by open-source language models such as GPT-NEO (Gao et al., 2021). This setup, where an adversary attempts to efficiently recover memorized training data, has been extensively studied on language models (Carlini et al., 2020; Zhang et al., 2021; Nasr et al., 2023). Other related work focuses on n-gram novelty versus copying from the training data (McCoy et al., 2023). Hartmann et al. (2023) and Zheng and Jiang (2022) provide surveys on types of memorization and their risks with respect to alignment, privacy and copyright. Importantly, we do not study any differences in model behavior on paragraphs within vs outside of the training data. This is another important privacy-related aspect known as Membership Inference Attack (Hu et al., 2021; Mattern et al., 2023; Shi et al., 2023).

Language Model Interpretability. Beyond identifying "what" training set paragraphs are memorized, we are interested in interpreting "how" a model does so. Chang et al. (2023) test whether different localization methods agree when localizing memorization in language models. The studied methods include brute-force zeroing out of model weights, learning a mask to prune weights and removing weights based on gradient attribution. In this work, we predominantly focus on gradient-based attribution (Sundararajan et al., 2017; Du et al., 2023), but also draw inspirations from activation patching (Meng et al., 2022; Geva et al., 2023) which aims at localizing the memorization of few-token facts instead of paragraphs. Existing interpretability work (Chang et al., 2023; Haviv et al., 2023) studies shorter memorized text spans such as idioms, URLs or quotes, for which memorization may have a different definition than for 100-token paragraphs. In §5, we borrow methods for gradient-based attribution using a contrastive objective from Maini et al. (2023). While their work focuses on memorizing atypical training set examples in image classification, we adapt their methods to memorization of paragraphs in language models. Related to our "memorization head" in §6, Yu et al. (2023) identify a "memory head" which however plays a widely different role. It down-weights geographic knowledge in in-context QA tasks.

Model Editing and Unlearning. Hase et al. (2023) ask whether "localization inform[s] editing" and led us to confirm our localization of relevant model parameters by fine-tuning only those parameters in an unlearning and model editing setting. Similar to their findings, we observe that memo-

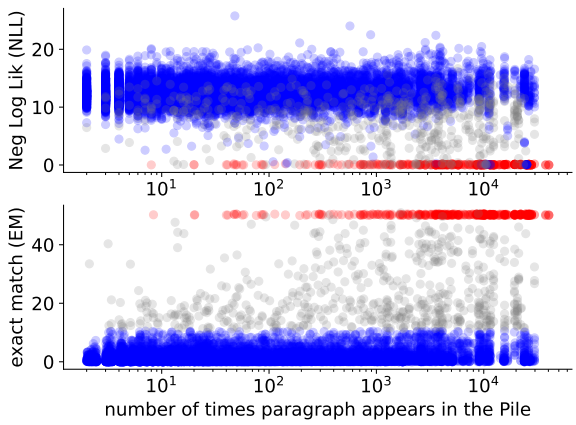

Figure 2: Splitting paragraphs of the PILE into memorized paragraphs and non-memorized paragraphs based on GPT-NEO 125M. We present the model with paragraph prefixes of length 50 tokens, greedy decode the next 50 tokens and evaluate the generation in terms of negative log-likelihood (NLL) and exact match (EM).

rization components are spread out across layers while patching-based methods in App. A.3 point to other components. Our model editing setup in §5.3 is similar to Eldan and Russinovich (2023), who find alternative paraphrases of facts that they use to fine-tune a model. Related areas of study are language model watermarking (Kirchenbauer et al., 2023) and grokking (Power et al., 2022).

3 Identifying Memorized Paragraphs

3.1 Open-Source Model and Training Set

GPT-NEO 125M. We seek to zoom in on a selected model to study its specific memorization behavior in detail. All presented methodology can however be transferred to any open-weight model. The GPT-NEO family of models (Gao et al., 2021) is intended to be the open-weight counterpart to the GPT-3 model (Brown et al., 2020) in terms of model architecture. GPT-NEO models are trained on a publicly available dataset, the PILE (Gao et al., 2021), which allows checking model generations against its training data. As such, they have been studied extensively with respect to how much they memorize (Carlini et al., 2022; Nasr et al., 2023). While these studies found that bigger model variants tend to memorize more, the smallest variant, GPT-NEO 125M, still exhibits extensive memorization behavior with an easier-to-study computational footprint. After all, when interpreting models at the level of individual weights, smaller models are easier to visualize and analyze.

The PILE. GPT-NEO 125M was trained on the PILE (Gao et al., 2021), an aggregation of 22 different datasets. It comprises 825GB of English text and code. For this study, we consider a post-processed 570GB subset of the PILE provided by Carlini et al. (2022). This subset contains 10,000 randomly sampled, unique 100-token paragraphs and the count how frequently they occur in the training set. We perform pre-processing steps to find a diverse set of paragraphs as detailed in App. A.1. This leaves us with 13,450 paragraphs of which the most frequent one occurs 40,382 times in the PILE.

3.2 Memorization Metrics and Data Split

We split the 13,450 PILE paragraphs $\mathcal{X}$ into a set of memorized paragraphs (MP) and non-memorized paragraphs (NMP) which are disjoint subsets $\mathcal{X} = \mathcal{X}^{\text{M}} \cup \mathcal{X}^{\text{NM}}$. To this end, we consider the exact match (EM) of the model's greedy decoding in an "extractable memorization" setting (Nasr et al., 2023). We also take the negative log-likelihood (NLL) into consideration.

Exact Match (EM). Exact match (EM) is the number of greedily decoded tokens that exactly match the tokens in the ground truth training set paragraph until the first mismatch. Since the continuations are 50 tokens long, EM $= 50$ is the maximum value.

Negative Log-Likelihood (NLL). Under a model with parameters $\boldsymbol{\theta}$, the negative log-likelihood for a batch of N paragraphs $x_{N,I}$ that are each I tokens long is given by $\mathcal{L}_{\text{NLL}}(\boldsymbol{x}_{N,I}; \boldsymbol{\theta}) = \frac{1}{N}\sum_n^N \Big(-\frac{1}{I}\sum_i^I \log p(x_{n,i} \mid \boldsymbol{x}_{n,0:i-1}; \boldsymbol{\theta})\Big)$. All paragraphs studied in this work are $I = 100$ tokens long of which the first 50 tokens are the prefix. We compute the NLL only on the last 50 (generated) tokens and omit the token position index i for simplicity in the following.

Memorized Paragraphs. Fig. 2 shows the NLL and EM results for all paragraphs. We select the 442 paragraphs with EM $= 50$ as the memorized set which is clearly distinct, both in terms of NLL and EM, from the other paragraphs. We provide an overview of some exemplary MPs in App. Tab. 1. Setting boundaries for a non-memorized set is less clear, but we choose the 12,422 paragraphs with $0 \leq$ EM ≤ 10. Similar to the EM $= 50$ paragraphs, those paragraphs form a distinctive cluster in Fig. 2. While there is high overlap when splitting based on NLL and EM, we observe that splitting

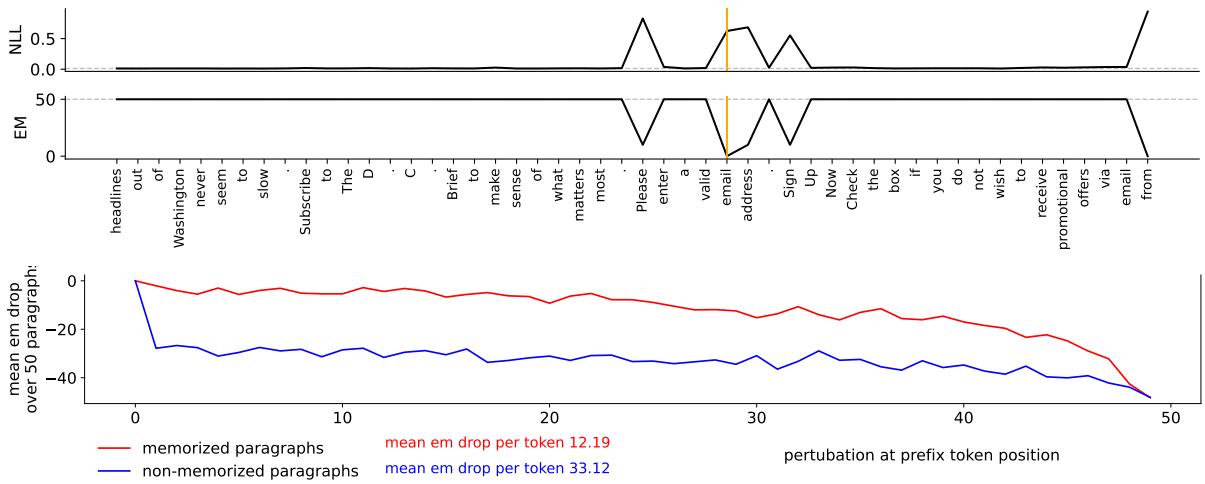

Figure 3: **[top]** The plot shows the effect of perturbing tokens in the prefix (shown) on the model's generation (not shown) in terms of the negative log-likelihood (NLL) and exact match (EM). Changing the single token "email" into a random other token causes the EM to drop by 45, even though "email" is about 20 tokens before the generated part. **[bottom]** Perturbing tokens in the memorized paragraphs has, on average, less impact in exact match drop (EM) in the model's generation, than perturbing tokens in the non-memorized paragraphs.

based on NLL yields less diverse, even more code-based examples since those generally have lower NLL. We hypothesize this is because there are less "second-best" paraphrases / alternatives for code.

4 Prefix Token Perturbation

Where in the paragraph do interventions disrupt memorization the most? We study this question by perturbing every token in the prefix, one token at a time, by replacing it with a random token from the vocabulary. For every 50-token prefix with a single perturbed token, we then use the language model to obtain a greedy decoding of the next 50 tokens. We measure the change in the decoding caused by the perturbation in terms of NLL and EM as shown at the top of Fig. 3. For different MPs, we often see that a few, distinctive tokens, even at early positions in the prefix, lead to a drop in EM of up to 45.

In Fig. 3 at the bottom, we zoom in on this finding by computing the mean EM drop per prefix token position over 50 MPs and NMPs. As expected, the closer the token to the decoded tokens (later in the prefix), the more impact the token has on the decoding. Interestingly, NMPs are, on average, easier perturbed than MPs. This may be hint at one property of memorization—MPs seem more "baked" into the model while NMPs with generally lower likelihood can easily "slip off" into equally likely paraphrases.

If a single token is able to divert the model's continuation of an MP, what does this continuation look like? The examples in Tab. 2 in the appendix demonstrate that the model's generations are syntactically and semantically mostly valid. In the following, we refer to those continuations based off a perturbed prefix as perturbed memorized paragraphs (PMPs). PMPs can be seen as admissible paraphrases of MPs.

5 Localizing Parameters

We investigate if there are any systematic differences in how the model internally processes our sets of MPs and NMPs. While we previously looked at token positions, we now turn to an analysis of model parameters which are shared across all token positions. Taking a simplified view, the model parameters are of shape $\boldsymbol{\theta} \in \mathbb{R}^{L \times C \times D^*}$, where $\{l\}_0^L$ indexes into the model's 12 layers, also known as Transformer blocks (Vaswani et al., 2017). For GPT-NEO 125M, each layer l consists of $C = 50$ model component types, $c \in \{\texttt{W_K H0}, \texttt{W_K H1}, \ldots\}$. The attention mechanism is comprised of 12 attention heads, each consisting of a key `W_K`, query `W_Q`, value `W_V`, and output `W_O` matrix. The multi-layer perceptron (MLP) block per layer consists of the input `W_in` and output matrix `W_out`. The layers and model components are shown on the Y and X axis in Fig. 4 respectively. D^* refers to the vector dimension, i.e., the number of weights which varies for each model component, thus, "D star" for simplicity.

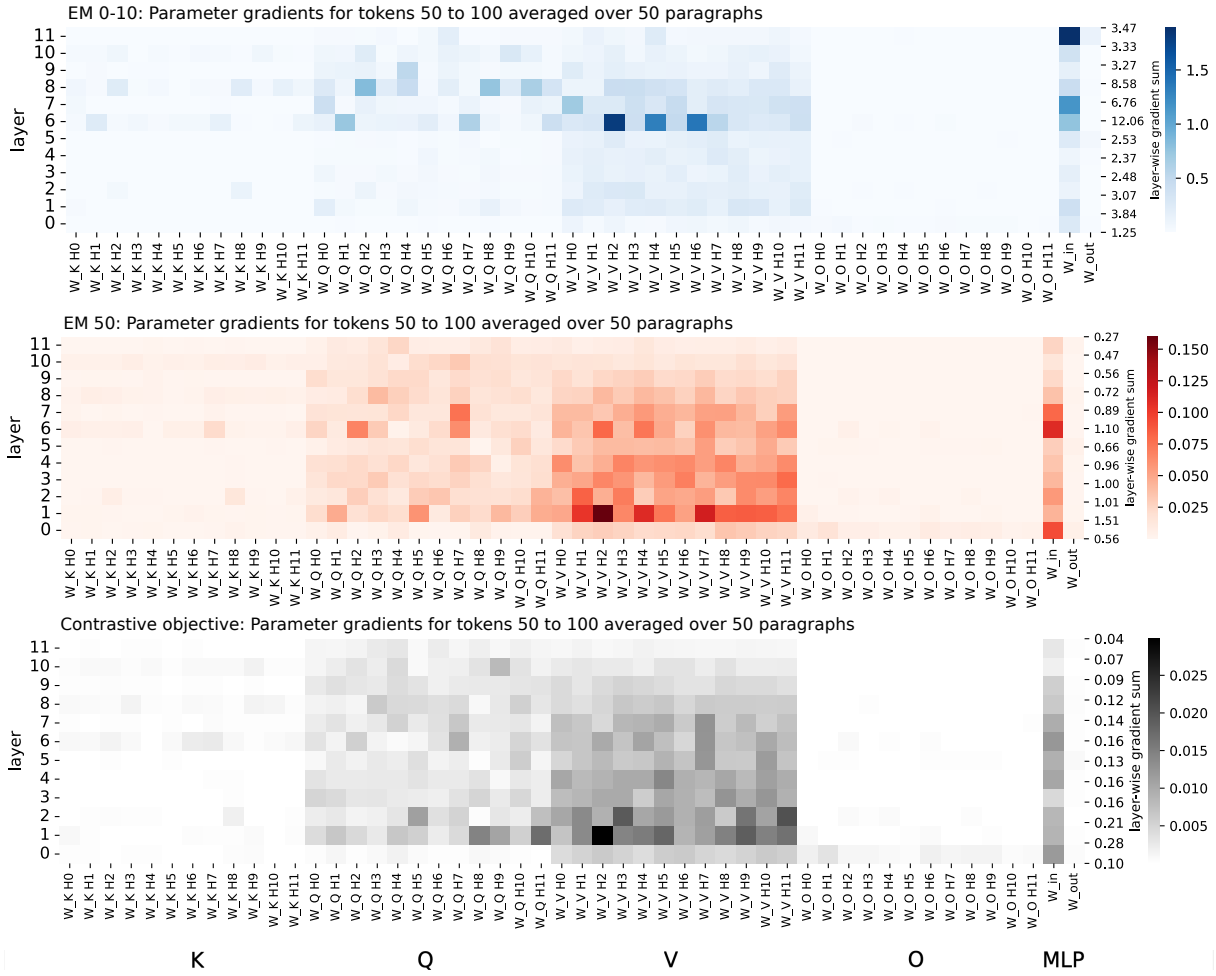

Figure 4: **[top and center]** While memorization appears to be spread across multiple layers, we observe systemically different parameter gradients for memorized and non-memorized paragraphs. The former is associated with lower absolute gradients in lower layers of the model. **[bottom]** Parameter gradient attribution scores for the contrastive objective (Eq. (3)).The value matrix (W_V) of attention head 2 in layer 1 appears to be strongly involved.

5.1 Gradient-based Parameter Attribution

We feed a batch of paragraphs to the language model and compute the NLL loss $\mathcal{L}_{\text{NLL}}$ for tokens 50 to 100, i.e., the generation of the model given the prefix. We then compute the parameter gradients $\Delta\boldsymbol{\theta} \in \mathbb{R}^{L \times C \times D^*}$ with respect to the loss:

$$\Delta\boldsymbol{\theta} = \frac{\partial \mathcal{L}_{\text{NLL}}(\boldsymbol{x}_N; \boldsymbol{\theta})}{\partial \boldsymbol{\theta}} \tag{1}$$

To obtain a *parameter gradient attribution score* $\Delta\theta_{l,c}$, we consider the absolute gradient value for all individual weights and choose the maximum value per layer l and component c:

$$\Delta\theta_{l,c} = \max_d \left(|\{\Delta\theta_{l,c,d}\}_d^{D^*}| \right) \tag{2}$$

In Fig. 4, we present the mean parameter gradient attribution scores for a batch of 50 MPs and, separately, a batch of 50 NMPs. We observe clear differences between the attribution scores: first of all, but less surprisingly, the gradients for the NMPs are larger since those are less likely under the model (Shi et al., 2023). More surprising are the clear differences with respect to layers: there is more gradient flow for MPs in lower layers, for both attention and MLP components, which is in line with Haviv et al. (2023). In fact, we observe a smooth shift in gradient patterns when evaluating "partly memorized" paragraphs with $10 \leq \text{EM} \leq 50$ as displayed in App. Fig. 9.

5.2 Contrastive Objective

Inspired by Chang et al. (2023)'s localization method, we combine MPs and NMPs in a contrastive objective. The objective is to change memorized continuations of MPs while preserving the model's continuations of NMPs, which translates

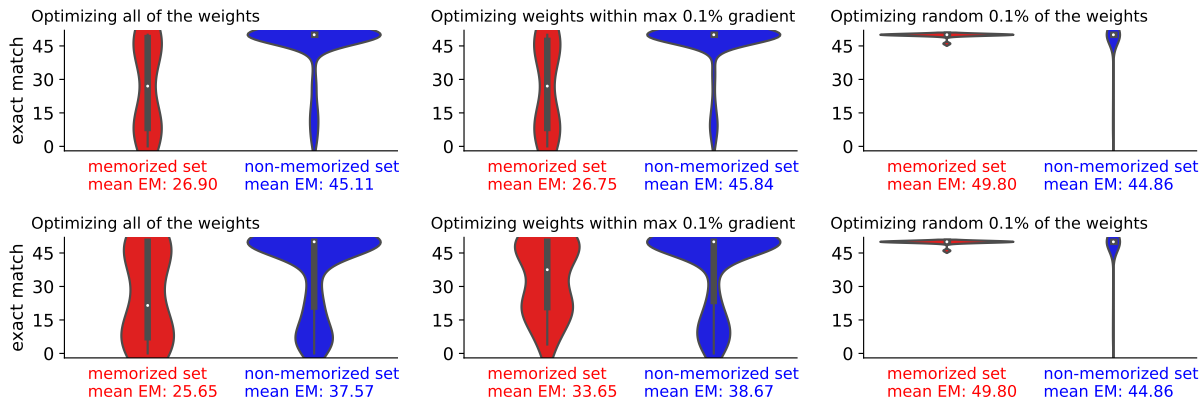

Figure 5: **[top]** To test whether our localization also informs editing, we optimize all model parameters based on the contrastive objective (Eq. (3)), only the 0.1% weights with the maximum gradient and a random sample of weights. Result shows that sparsely fine-tuning only the max gradient weights causes the most unlearning in MPs and the least in NMPs. **[bottom]** Instead of unlearning MPs, we consider an editing objective (Eq. (4)) to overwrite MPs using PMPs. While sparse optimization of only the max gradient weights appears to be similarly effective as training all weights, editing is overall more difficult than unlearning.

into the following contrastive objective (CO):

$$\mathrm{CO}_{\downarrow}(\boldsymbol{x}_n^{\mathrm{M}}, \boldsymbol{x}_N^{\mathrm{NM}}; \boldsymbol{\theta}) = -\mathcal{L}_{\mathrm{NLL}}(\boldsymbol{x}_n^{\mathrm{M}}; \boldsymbol{\theta}) + \mathcal{D}_{\mathrm{KL}}\big((\boldsymbol{x}_N^{\mathrm{NM}}; \boldsymbol{\theta}), (\boldsymbol{x}_N^{\mathrm{NM}}; \boldsymbol{\theta}_0)\big) \tag{3}$$

The CO increases the NLL of an individual MP $\boldsymbol{x}_n^{\mathrm{M}}$ and decreases the KL divergence $\mathcal{D}_{\mathrm{KL}}$ from the model's original continuations of a batch of N NMPs $\boldsymbol{x}_N^{\mathrm{NM}}$. This set of NMPs can be seen as a "control" set that ensures the model remains as much as possible unaltered. We denote $\boldsymbol{\theta}_0$ as the model's original parameters which are excluded (detached) from the gradient computation. To study the removal of multiple MPs, we recompute the CO over 50 different MPs and randomly sampled batches of NMPs and aggregate all gradient computations. We rely on `TransformerLens` (Nanda, 2023) for the implementation of this and the following experiments. We disable gradient computation on the model components "`embed`", "`pos_embed`", "`unembed`" and all bias terms. As shown in Fig. 4, the parameter gradient attribution scores yield by the contrastive objective reveal similar patterns to those observed in Fig. 4. Most importantly, in both settings, the value matrix (`W_V`) of attention head 2 in layer 1 is most salient.

5.3 Sparse Unlearning and Editing

Instead of computing gradients to only obtain attribution scores, we may also update the model parameters based on the gradients in an optimization setting to satisfy the contrastive objective (CO) in Eq. (3). This can help us find further evidence that the localized parameters are meaningful for memorization (Hase et al., 2023).

Unlearning MPs. We compute the gradients of all parameters with respect to the CO and mask out all parameters that are not within the maximum 0.1 % of all absolute gradient values. We keep this mask while taking 10 gradient steps using the Adam optimizer (Kingma and Ba, 2015) which can be seen as a form of sparse fine-tuning. We compare this setting against optimizing all of the weights and masking a random 0.1 % of the weights as shown in Fig. 5. While the goal is to bring down the EM of MPs from formerly 50 to 0, the EM of the model's original continuation of the NMPs should remain unchanged (EM = 50). We find that the result between optimizing all weights versus only the 0.1% max gradient weights does not worsen. To the contrary, there is even more drop in EM on the MPs and less drop on the NMPs. Moreover, optimizing a randomly selected 0.1% of weights does not achieve the desired result at all.

Editing MPs into PMPs. Instead of "unlearning" MPs, we make an effort to edit them into PMPs with a modified contrastive objective:

$$\mathrm{CO}_{\leftrightarrow}(\boldsymbol{x}_n^{\mathrm{M}}, \boldsymbol{x}_N^{\mathrm{NM}}; \boldsymbol{\theta}) = +\mathcal{L}_{\mathrm{NLL}}(\boldsymbol{x}_n^{\mathrm{M}}; \boldsymbol{\theta}) + \mathcal{D}_{\mathrm{KL}}\big((\boldsymbol{x}_N^{\mathrm{NM}}; \boldsymbol{\theta}), (\boldsymbol{x}_N^{\mathrm{NM}}; \boldsymbol{\theta}_0)\big) \tag{4}$$

Instead of increasing the NLL on MPs $\boldsymbol{x}_n^{\mathrm{M}}$, we are now decreasing the NLL on PMPs $\boldsymbol{x}_n^{\mathrm{M}}$ to make their alternative continuations more likely. The editing results for 10 optimization steps is presented

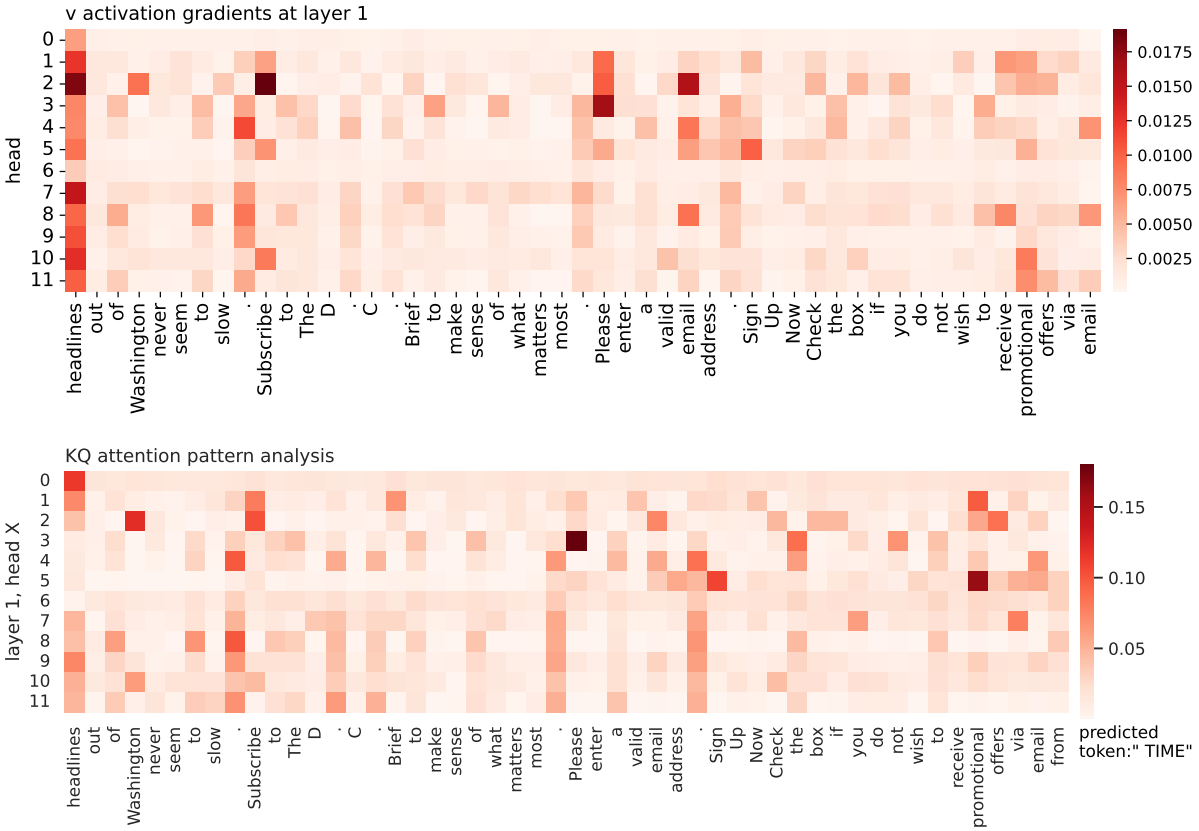

Figure 6: **[Top]** Value activation gradients on layer 1. **[Bottom]** KQ attention on layer 1. We find that head 2 shows similar attention patterns in both, **[Top]** and **[Bottom]**: more distinctive tokens such as "Washington", "Subscribe" or "email" are more influential and are often the ones causing most perturbation to memorized paragraphs (§4).

in Fig. 5. Again, optimizing only a masked 0.1% of high gradient weights performs equally well to optimizing all weights. Comparing results however suggests that unlearning is easier than editing. A common finding from perturbing the prefix (Fig. 3), unlearning and editing MPs (Fig. 5) is that it is indeed difficult to remove MPs while leaving NMPs unchanged.

6 Memorization Head L1H2

In §5, different analysis methods point to the same model component, the value matrix of attention head 2 in layer 1. This is in line with Haviv et al. (2023) who find that memorized tokens are promoted in lower layers and it motivates us to study the role of this specific head in more detail.

6.1 Activation Gradients

Instead of computing gradients with respect to parameters as in Eq. (1), we now compute gradients with respect to activations $\boldsymbol{h} \in \mathbb{R}^{L \times C \times I \times D^*}$:

$$\Delta \boldsymbol{h} = \frac{\partial \mathcal{L}_{\text{NLL}}(\boldsymbol{x}_N; \boldsymbol{h})}{\partial \boldsymbol{h}} \tag{5}$$

As before, we consider absolute gradients and max-pool over the (hidden) dimension D^* to obtain attribution scores $\Delta h_{l,c,i}$ per layer l, model component c and token position i. Fig. 6 [top] shows the value activation attribution scores for layer 1 for an exemplary MP. Again, head 2 appears to be particularly active and somewhat anti-correlated with the other heads. For instance, head's 2 gradient attribution is large for the tokens "Subscribe" or "Washington", and not for their neighboring tokens "." or "of" as most other heads. Interestingly, these tokens also seem distinctive / descriptive for the given paragraph and the token "email" which caused most perturbation in Fig. 3 is standing out.

6.2 Activation Pattern Analysis

We observe similar patterns when analyzing forward pass activations of *key-query (KQ) attention patterns*. The normalized, inner product of "keys" $\boldsymbol{k}$ and "queries" $\boldsymbol{q}$ is given by $\text{softmax}(\boldsymbol{kq})$ and describes the amount of "lookback" attention from the currently decoded token to all previous tokens. In our case, we choose to study the attention be-

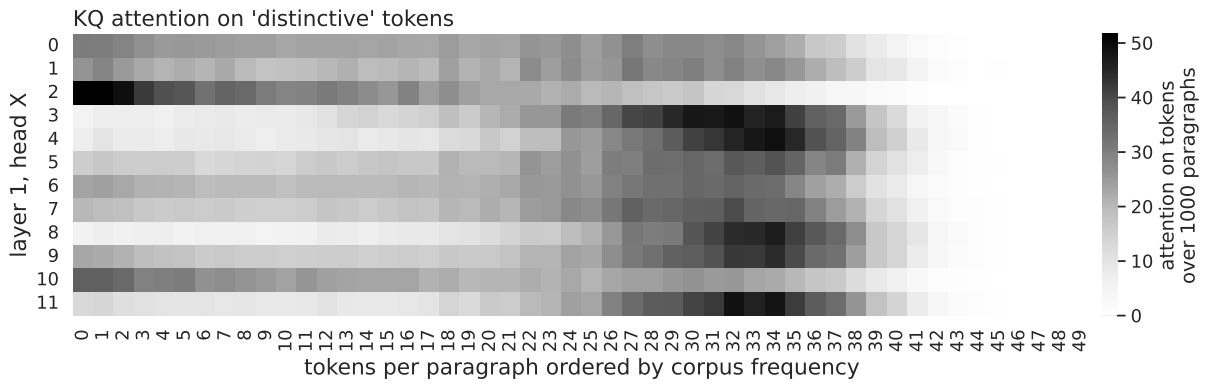

Figure 7: The *memorization head* 2 in layer 1 is strongly negatively correlated (-0.97) with the corpus-level frequency of tokens. The plot shows the aggregated attention that each head assigns to tokens per paragraph ranked by corpus frequency. Note that, due to ties in token frequencies, often not all ranks up to rank 49 receive attention.

tween the first decoded token onto the full 50-token prefix as shown in Fig. 6 [bottom]. Similar to the activation gradients, head 2 attends to seemingly distinctive or rare tokens such as "Subscribe", "Washington", "email" or "offers" instead of more frequent tokens like punctuation marks and stop words as heads 3 to 11 do. Recent work (Tigges et al., 2023; Sun et al., 2024) finds that punctuation marks often serve as "aggregation points" for sentiment throughout a paragraph. It is important to note that these attention patterns per head look entirely different for any other layer, such as layer 2 visualized in Fig. 12 in the appendix.

6.3 Rare Token Correlation

When perturbing tokens (§4), and analyzing activations (§6.1, §6.2), we find that "rare" tokens play an important role for memorization, related to other previous findings on the relation between long tail distributions and memorization (Feldman and Zhang, 2020). To test this *rate token hypothesis*, we consider the unigram distribution of all tokens in our corpus which amounts to 34,562 unique tokens. For every paragraph in our corpus, we rank the tokens by their corpus frequency from 0 (most rare) to 49 (most frequent) allowing ties. Then, we feed each paragraph to GPT-NEO 125M, obtain the KQ attention of the first decoded token at onto every prefix token. We go through the paragraph's token frequency ranks and sum up the attention that each head assigns to the token of each rank. As shown in Fig. 7, we find that head number 2 in layer 1 is indeed the one most strongly correlated with rare tokens. As such, we have identified an important function of a model component that plays a vital role in memorizing paragraphs. One may hypothesize that the model computes a signature of each paragraph as a "bag of its rare words". It could then use this signature as a query to look up its "memory of paragraphs" seen during training.

7 Discussion

Our focus lies on identifying "where" memorization-relevant model components may be localized, but our findings open up interesting follow-up questions on the "why" and "how". In §5.3, we are unlearning and editing MPs, but memorization may similarly lead to better performance or may be desired for certain types of paragraphs (Feldman and Zhang, 2020). One could in fact take an opposite view and study how to make a model memorize an NMP. Being able to identify differences in the model-internal processing of MPs and NMPs, future work could train a classifier on the activations or gradients (Pimentel et al., 2022; Li et al., 2023) to detect looming memorization at decoding time instead of considering logit distributions or post-hoc string matching (Shi et al., 2023). Similar to our token perturbations in §4, future work could attempt to divert memorized continuations through targeted interventions in the forward pass.

8 Conclusion

Gradients flow differently for memorized (more in lower layers) than for non-memorized paragraphs (more in higher layers). While many model components are involved, memorization is often localized to few, distinctive tokens in the prefix that are predominantly processed by the attention head 2 in layer 1 of GPT-NEO 125M.

Acknowledgments

We would like to thank the Google AI Developer Assistance team (AIDA) as well as Katherine Lee, Neel Nanda, Nicholas Carlini, Timo Denk, Richard Shin, Xiang Deng, Bin Ni, Alex Polozov, Luca Beurer-Kellner and Suchin Gururangan, and Mengzhou Xia.

Limitations

The purpose of this work is to study paragraph memorization of one model in detail. Our methodology is however not model-specific and can be applied to other models such as the Pythia family (Biderman et al., 2023). Another important direction is memorization in instruction- and RLHF-tuned models. Most prior work (Carlini et al., 2020, 2022; McCoy et al., 2023) and our paper identify memorization through prefix continuation, but instruction-tuned models may behave and memorize entirely differently. Importantly, there are ongoing discussions on the explanatory value of gradients (Sundararajan et al., 2017; Du et al., 2023) and activations (Farquhar et al., 2023; Stoehr et al., 2024). By combining different interpretability methods such as analyses of parameter gradients, activation gradients, token perturbation and patching, we make an effort to provide different perspectives and find that different methods point to similar model components and mechanisms.

Impact Statement

Language model memorization has important implications with respect to performance, copyright and privacy concerns. To limit risks, we specifically study a small, open-weight model GPT-NEO 125M and a widely studied public training set. We hope that a better understanding of memorization can help improve model performance and promotes the open-sourcing of language models. Not wanting to publicly leak organization-internal data or risking copyright infringement is a primary blocker for open-source efforts.

References

Stella Biderman, Hailey Schoelkopf, Quentin Anthony, Herbie Bradley, Kyle O'Brien, Eric Hallahan, Mohammad Aflah Khan, Shivanshu Purohit, USVSN Sai Prashanth, Edward Raff, Aviya Skowron, Lintang Sutawika, and Oskar van der Wal. 2023. Pythia: A suite for analyzing large language models across training and scaling. *arXiv*, 2304.01373.

Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel Ziegler, Jeffrey Wu, Clemens Winter, Chris Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. 2020. Language models are few-shot learners. In *Advances in Neural Information Processing Systems*, volume 33, pages 1877–1901.

Nicholas Carlini, Daphne Ippolito, Matthew Jagielski, Katherine Lee, Florian Tramer, and Chiyuan Zhang. 2022. Quantifying memorization across neural language models. *ICLR*.

Nicholas Carlini, Florian Tramer, Eric Wallace, Matthew Jagielski, Ariel Herbert-Voss, Katherine Lee, Adam Roberts, Tom Brown, Dawn Song, Ulfar Erlingsson, Alina Oprea, and Colin Raffel. 2020. Extracting training data from large language models. *arXiv*, 10.48550.

Ting-Yun Chang, Jesse Thomason, and Robin Jia. 2023. Do localization methods actually localize memorized data in llms? ArXiv:2311.09060 [cs].

Kevin Du, Lucas Torroba Hennigen, Niklas Stoehr, Alex Warstadt, and Ryan Cotterell. 2023. Generalizing backpropagation for gradient-based interpretability. In *ACL*, pages 11979–11995, Toronto, Canada.

Ronen Eldan and Mark Russinovich. 2023. Who's Harry Potter? Approximate unlearning in LLMs. ArXiv:2310.02238 [cs].

Sebastian Farquhar, Vikrant Varma, Zachary Kenton, Johannes Gasteiger, Vladimir Mikulik, and Rohin Shah. 2023. Challenges with unsupervised LLM knowledge discovery. *arXiv*, 2312.10029.

Vitaly Feldman and Chiyuan Zhang. 2020. What neural networks memorize and why: discovering the long tail via influence estimation. *NeurIPS*.

Leo Gao, Stella Biderman, Sid Black, Laurence Golding, Travis Hoppe, Charles Foster, Jason Phang, Horace He, Anish Thite, Noa Nabeshima, Shawn Presser, and Connor Leahy. 2021. The Pile: An 800Gb dataset of diverse text for language modeling. *arXiv*, 2101.00027.

Mor Geva, Jasmijn Bastings, Katja Filippova, and Amir Globerson. 2023. Dissecting recall of factual associations in auto-regressive language models. *arXiv*, 2304.14767.

Valentin Hartmann, Anshuman Suri, Vincent Bindschaedler, David Evans, Shruti Tople, and Robert West. 2023. Sok: memorization in general-purpose large language models. ArXiv:2310.18362 [cs].

Peter Hase, Mohit Bansal, Been Kim, and Asma Ghandeharioun. 2023. Does localization inform editing? Surprising differences in causality-based localization vs. knowledge editing in language models. *NeurIPS*.

Adi Haviv, Ido Cohen, Jacob Gidron, Roei Schuster, Yoav Goldberg, and Mor Geva. 2023. Understanding transformer memorization recall through idioms. *EACL*.

Hongsheng Hu, Zoran Salcic, Lichao Sun, Gillian Dobbie, Philip S. Yu, and Xuyun Zhang. 2021. Membership inference attacks on machine learning: A survey. *ACM Computing Surveys*.

Diederik Kingma and Jimmy Ba. 2015. Adam: A method for stochastic optimization. In *International Conference on Learning Representations*, page 337.

John Kirchenbauer, Jonas Geiping, Yuxin Wen, Jonathan Katz, Ian Miers, and Tom Goldstein. 2023. A watermark for large language models. *ICML*.

Kenneth Li, Oam Patel, Fernanda Viégas, Hanspeter Pfister, and Martin Wattenberg. 2023. Inference-time intervention: Eliciting truthful answers from a language model. *NeurIPS*.

Pratyush Maini, Michael C. Mozer, Hanie Sedghi, Zachary C. Lipton, J. Zico Kolter, and Chiyuan Zhang. 2023. Can neural network memorization be localized? *ICML*.

Justus Mattern, Fatemehsadat Mireshghallah, Zhijing Jin, Bernhard Schoelkopf, Mrinmaya Sachan, and Taylor Berg-Kirkpatrick. 2023. Membership inference attacks against language models via neighbourhood comparison. In *Findings of ACL*, pages 11330–11343.

Thomas McCoy, Paul Smolensky, Tal Linzen, Jianfeng Gao, and Asli Celikyilmaz. 2023. How much do language models copy from their training data? Evaluating linguistic novelty in text generation using raven. *Transactions of the Association for Computational Linguistics*, 11:652–670.

Kevin Meng, David Bau, Alex Andonian, and Yonatan Belinkov. 2022. Locating and editing factual associations in GPT. *NeurIPS*.

Neel Nanda. 2023. TransformerLens—A library for mechanistic interpretability of generative language models.

Milad Nasr, Nicholas Carlini, Jonathan Hayase, Matthew Jagielski, A. Feder Cooper, Daphne Ippolito, Christopher A. Choquette-Choo, Eric Wallace, Florian Tramèr, and Katherine Lee. 2023. Scalable extraction of training data from (production) language models.

New York Times. 2023. One hundred examples of GPT-4 memorizing content from the New York Times, Document 1-68, Exhibit J.

Koyena Pal, Jiuding Sun, Andrew Yuan, Byron C. Wallace, and David Bau. 2023. Future Lens: Anticipating subsequent tokens from a single hidden state. *CoNLL*.

Tiago Pimentel, Josef Valvoda, Niklas Stoehr, and Ryan Cotterell. 2022. The architectural bottleneck principle. In *Conference on Empirical Methods in Natural Language Processing (EMNLP)*.

Alethea Power, Yuri Burda, Harri Edwards, Igor Babuschkin, and Vedant Misra. 2022. Grokking: Generalization beyond overfitting on small algorithmic datasets. *arXiv*, 2201.02177.

Weijia Shi, Anirudh Ajith, Mengzhou Xia, Yangsibo Huang, Daogao Liu, Terra Blevins, Danqi Chen, and Luke Zettlemoyer. 2023. Detecting pretraining data from large language models. *arXiv*, 2310.16789.

Niklas Stoehr, Pengxiang Cheng, Jing Wang, Daniel Preotiuc-Pietro, and Rajarshi Bhowmik. 2024. Unsupervised contrast-consistent ranking with language models. *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics*.

Mingjie Sun, Xinlei Chen, J. Zico Kolter, and Zhuang Liu. 2024. Massive activations in large language models. *arXiv*, 2402.17762.

Mukund Sundararajan, Ankur Taly, and Qiqi Yan. 2017. Axiomatic attribution for deep networks. In *ICML*, pages 3319–3328. Event-place: Sydney, NSW, Australia.

Curt Tigges, Oskar John Hollinsworth, Atticus Geiger, and Neel Nanda. 2023. Linear representations of sentiment in large language models. *arXiv*, 2310.15154.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In *NeurIPS*.

Qinan Yu, Jack Merullo, and Ellie Pavlick. 2023. Characterizing mechanisms for factual recall in language models. *arXiv*, 2310.15910.

Chiyuan Zhang, Daphne Ippolito, Katherine Lee, Matthew Jagielski, Florian Tramèr, and Nicholas Carlini. 2021. Counterfactual memorization in neural language models. *NeurIPS*.

Fred Zhang and Neel Nanda. 2024. Towards best practices of activation patching in language models: Metrics and methods. In *ICLR*.

Xiaosen Zheng and Jing Jiang. 2022. An empirical study of memorization in NLP. In *ACL*, pages 6265–6278, Dublin, Ireland.

A Appendix

A.1 Paragraph Pre-Processing

We filter out paragraphs containing any variation of keywords that are disproportionally frequent in Carlini et al. (2022)'s PILE subset: "TripAdvisor, href, license, copyright, software, manuscript, submission, distribution, disclaimed, limited". As a second preprocessing step, we filter out all paragraphs that contain less than 50% of unique tokens to remove paragraphs containing mostly white spaces.

A.2 Activation Analysis at Selected Tokens

In §4, we perturb single tokens in the prefix and measure the incurred change in the model's continuation with respect to the originally, memorized continuation. We then pick the token position that causes the maximum change and term it the *perturbed token*. In the model's generation, we pick the first token that is changed with respect to the unperturbed continuation and call it the *impact token*. Next, we pass both paragraphs, the memorized paragraph and the perturbed memorized paragraph, to the model and compute the activation gradients at the perturbed and the impact token following §6.1. The result in Fig. 10 shows large gradients for key and query activations at layer 2. At the impacted token, query activation gradients are generally more active.

A.3 Patching-based Attribution

In addition to studying activation gradients at the perturbed and impacted token, we experiment with activation patching (Meng et al., 2022; Pal et al., 2023; Zhang and Nanda, 2024). We either consider perturbed memorized paragraphs as the *clean run* and patch in activations at the perturbed token position from the memorized paragraphs or vice versa. As a patching metric, we measure the change in NLL at the first impacted token. The results for 50 different paragraphs are presented in Fig. 11.

NLL	EM	paragraph
0.426	50	;Classes</a></li> </ul> <script type="text/javascript"><!– allClassesLink = document.getElementById("allclasses_navbar_top"); if...
1.074	50	headlines out of Washington never seem to slow. Subscribe to The D.C. Brief to make sense of what matters most. Please enter a valid...
0.387	50	Sign up for Take Action Now and get three actions in your inbox every week. You will receive occasional promotional offers for ...
0.259	50	The following are trademarks or service marks of Major League Baseball entities and may be used only with permission of Major League...
0.276	50	0" SUMMARY="">\n<TR>\n<TD COLSPAN=2 BGCOLOR="#EEEEFF" CLASS="NavBarCell1">\n<A NAME="navbar_top_firstrow"...

Table 1: Representative paragraphs that are memorized by GPT-NEO 125M based on the exact match (EM) of 50 tokens between the model's generation and the ground truth training set paragraph.

prefix	**memorized continuation**	**perturbed continuation**
></li>\n</ul>\n \n<script type="text/javascript"><!– \n allClassesLink = document.getElementById("allclasses_navbar_bottom");\n if(window==top	) {\n allClassesLink.style.display = "block";\n }\n else {\n allClassesLink.style.display = "none";\n }\n //–>\n</script	= "/ top";\n visibility = { Tosink: "visibility" };\n adaption = document.getElementById("adaption");\n if(idaption)\n aada
headlines out of Washington never seem to slow. Subscribe to The D.C. Brief to make sense of what matters most. Please enter a valid email address. Sign Up Now Check the box if you do not wish to receive promotional offers via email from	TIME. You can unsubscribe at any time. By signing up you are agreeing to our Terms of Use and Privacy Policy . This site is protected by reCAPTCHA and the Google Privacy Policy and Terms of Service apply. Thank you! For your	icon.com/dccbrief./\n/\nThe D.C. Brief is a weekly newsletter focused on the latest news and information from the Washington, D.C. area. Sign up here./\n/\nThe D.C. Brief
Sign up for Take Action Now and get three actions in your inbox every week. You will receive occasional promotional offers for programs that support The Nation's journalism. You can read our Privacy Policy here. Sign up for Take Action Now and get	three actions in your inbox every week.\n\nThank you for signing up. For more from The Nation, check out our latest issue\n\nSubscribe now for as little as $2 a month!\n\nSupport Progressive Journalism The Nation is reader supported:	03 in your inbox every week.\n\nThank you for signing up. For more from The Nation, check out our latest issue\n\nSubscribe now for as little as $2 a month!\n\nSupport Progressive Journalism The Nation is reader supported:
The following are trademarks or service marks of Major League Baseball entities and may be used only with permission of Major League Baseball Properties, Inc. or the relevant Major League Baseball entity: Major League, Major League Baseball, MLB, the silhouetted batter logo	, World Series, National League, American League, Division Series, League Championship Series, All-Star Game, and the names, nicknames, logos, uniform designs, color combinations, and slogans designating the Major League Baseball clubs and entities, and	and/or othershaped breakdown may loom, featuring provisions of the Koehlers and/or its logo, and may include design events or other references to the Koehlers and/or their logo.\n\nYamaha,
0" SUMMARY="">\n<TR>\n<TD COLSPAN=2 BGCOLOR="#EEEEFF" CLASS="NavBarCell1">\n<A NAME="navbar/_top/_firstrow"><!– –	></A>\n<TABLE BORDER="0" CELLPADDING="0" CELLSPACING="3" SUMMARY="">\n <TR ALIGN="center" VALIGN="top">\n <TD	TD>\n<TD VALIGN=3><FONT SIZE="-2">\n PREV NEXT</FONT ></TD>\n<TD VALIGN=3><FONT SIZE="

Table 2: In §4, we perturb tokens in the prefix **[left]** and check how the model's perturbed continuations **[right]** changes with respect to the original, memorized continuation **[center]**. We find that perturbed continuations are still largely syntactically and semantically valid.

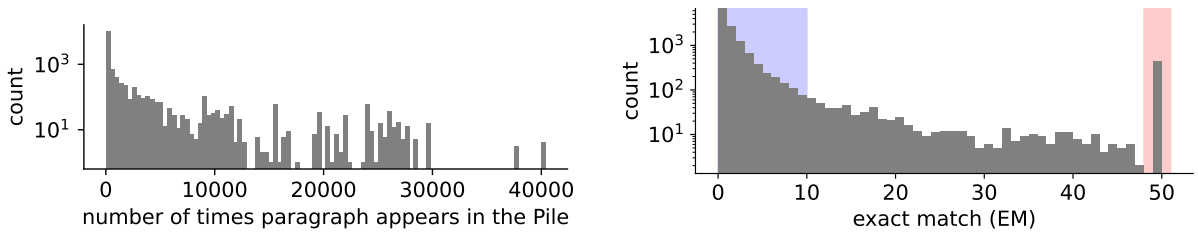

Figure 8: **[left]** Frequency count of each paragraph in our PILE subset borrowed from Carlini et al. (2022). **[right]** Exact match (EM) distribution for all paragraphs in our dataset. We consider paragraphs with EM $= 50$ as memorized and $0 \leq$ EM ≤ 50 as non-memorized.

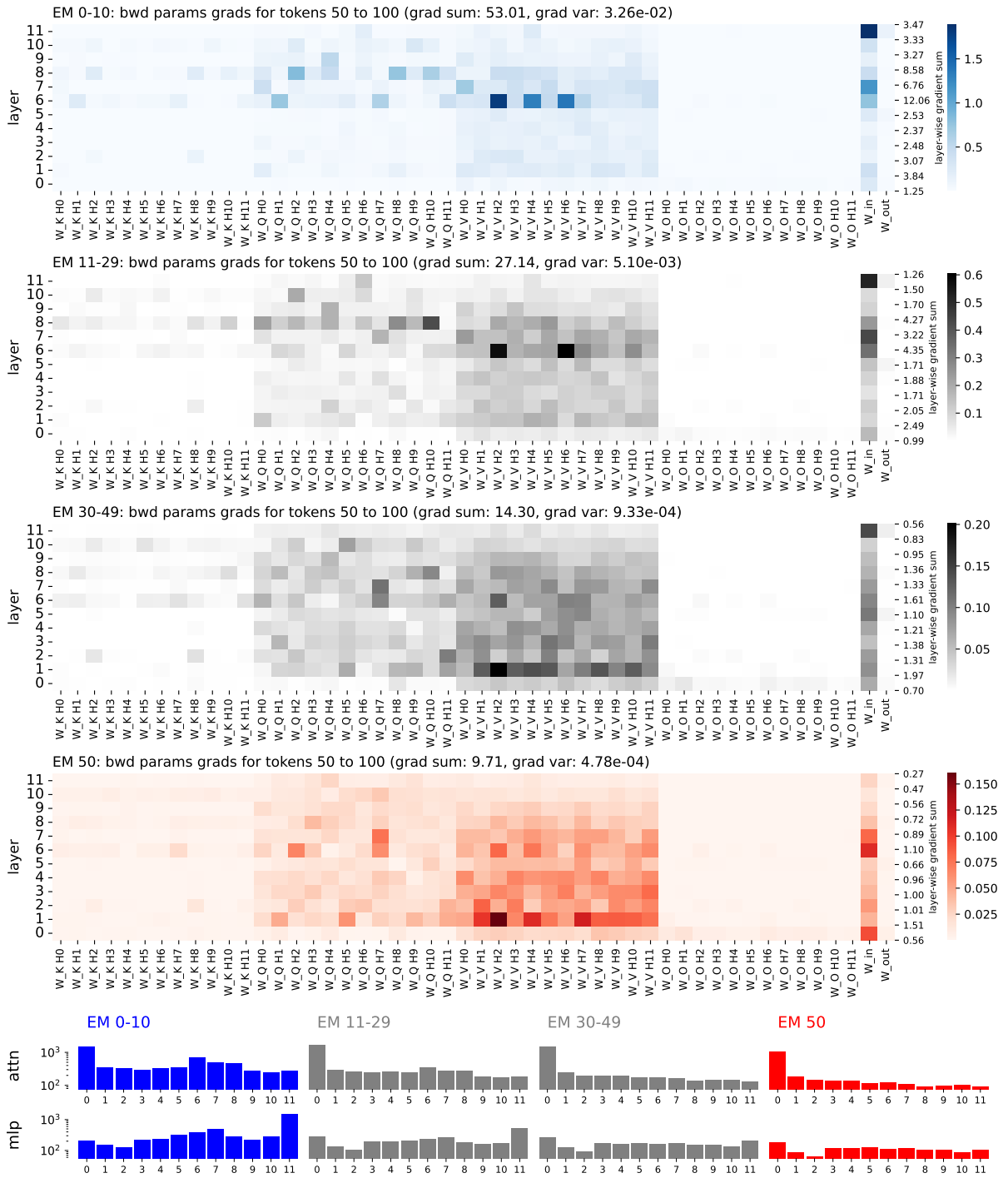

Figure 9: Supplementary plot to Fig. 4 showing the parameter gradients for paragraphs with $0 \leq \text{EM} \leq 10$, $11 \leq \text{EM} \leq 29$, $30 \leq \text{EM} \leq 49$ and $\text{EM} = 50$. The bar plots visualize the absolute gradient sums for all attention (attn) and multi-layer perceptron (mlp) blocks. For memorized paragraphs, we observe that overall gradient flower is less and lower layers tend to have higher gradients. This change in gradient patterns from non-memorized paragraphs to memorized paragraphs appears to be smooth.

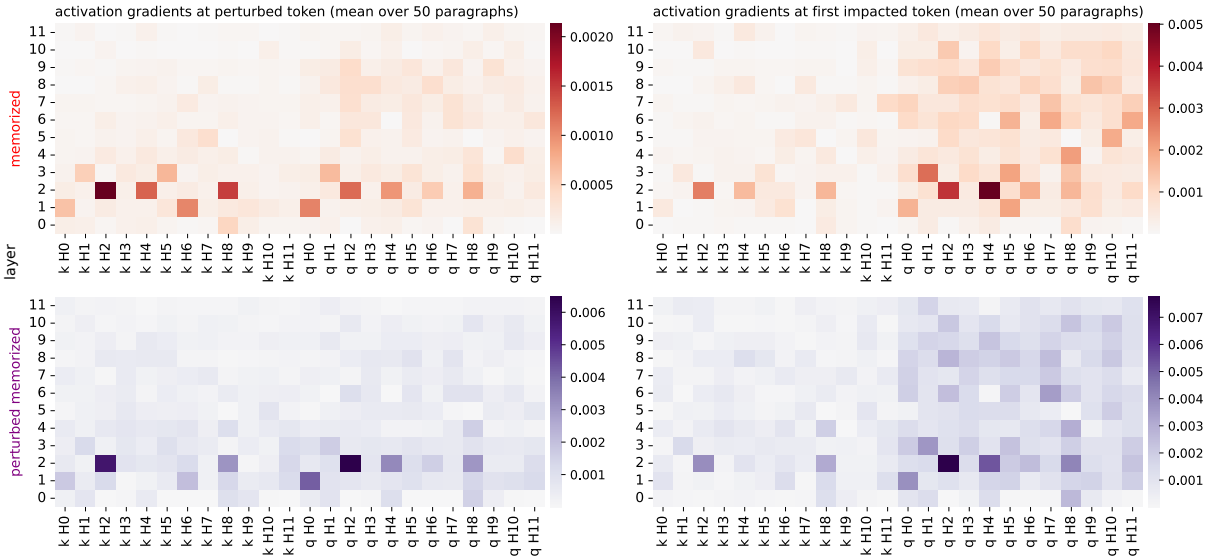

Figure 10: Comparing activation gradients of 50 memorized paragraphs and their perturbed memorized counterparts at the perturbed token and the first impacted token.

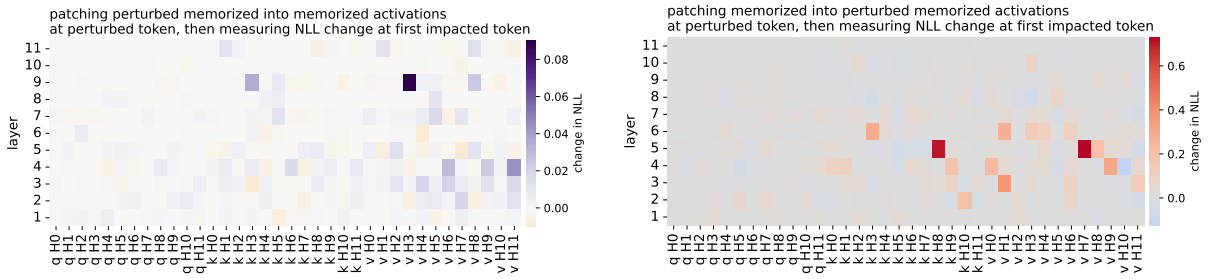

Figure 11: "Two-way activation patching" at the perturbed token to identify the change in the impacted token.

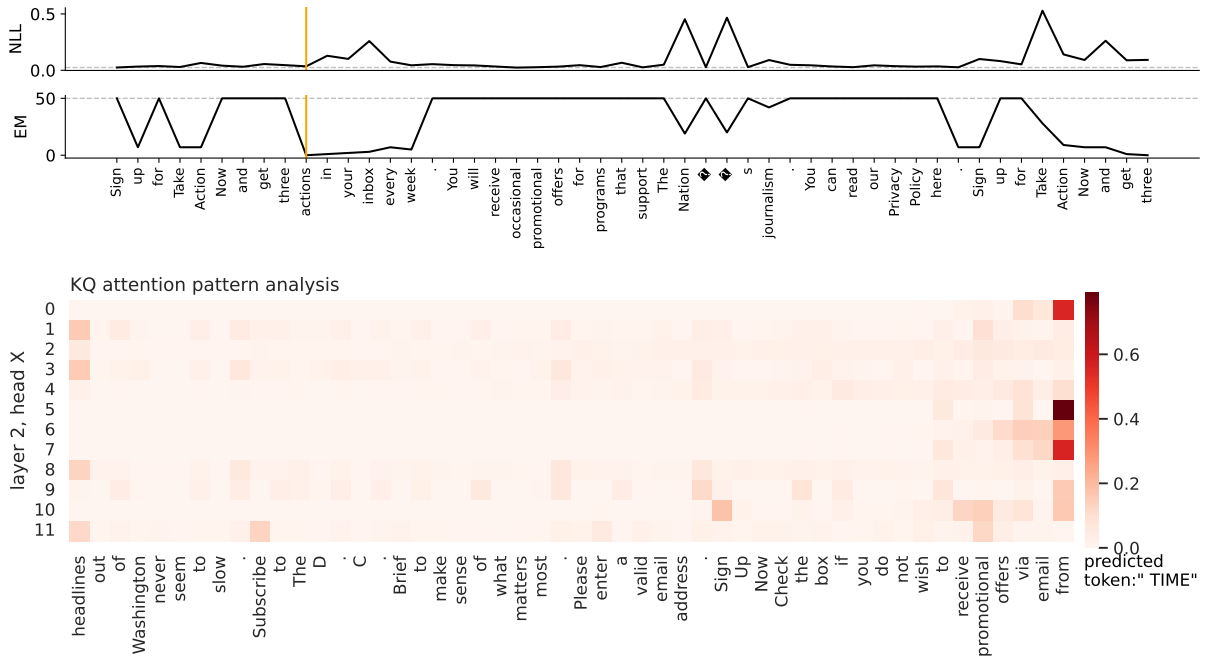

Figure 12: **[top]** Another example of perturbing the prefix tokens of a memorized paragraph as presented in Fig. 3. **[bottom]** Analysis if KQ attention patterns on layer 2 to compare against patterns in layer 1 presented in Fig. 6.